Part 1:

1. Row Key & Sharding:
   The proposed Row Key is *[State]#[County]#[City]#[ModelYear]#[VIN]*. While prepending a hash to this key would cause writes to be evenly distributed across all nodes and prevent write hotspots, I am choosing this geographic range-based key to allow efficient prefix scans. If I were to prepend a hash, it would impact my ability to efficiently query vehicles by City or Make because the hashed data would require N parallel prefix scans. Fast parallel reads would still be possible, but it would increase the application's complexity.
2. Column Families:
   I would have three column families: *vehicle_specs(Make, Model, EV_Type, Electric_Range, Base_MSRP), geo_location(Postal_Code, Legislative_District, Vehicle_Location_Coordinates), and admin_data(DOL_Vehicle_ID, CAFV_Eligibility)*. This grouping would aid in distributed read performance because queries that only need geographic data would only read the *location* family from disk, thereby, bypassing the disk I/O and network overhead that would come from reading the specs or registration data.
3. Horizontal Scaling:
   The advantages in using a Hashed Shard Key over a Range-Based Shard Key is that the write throughput is at its most efficient since inserts are randomly distributed across the cluster but it is inefficient and expensive for range queries. On the other hand, Range-Based Shard Key is very efficient because it minimizes network latency by reading contiguous data from a single shard during range queries, but creates situations where new inserts overwhelm a single shard and the rest stay idle.

Part 4:

According to my performance tests, the fast write averaged around 33.32 ms while the safe write averaged around 35.55 ms. Based on the CAP theorem, this demonstrates the trade-off between availability and consistency. A real-world scenario where I would choose the faster write, despite the risk of data loss, is keeping track of the number of views/likes on a social media post. Since posts can have thousands of other users interacting with it in many different ways, it is incredibly important for the database to have high write throughput to avoid crashing. If a failure occurs on a primary node and some views/likes are lost, before secondary nodes are able to be replicated, the impact it will have on the user is not severe. Therefore, in this case, it is alright to sacrifice consistency for speed.